

PalmSource®

Conference and Expo

PalmSource®
Conference and Expo



New ARM-based Palm OS APIs

David Fedor, architect/evangelist
Owen Emry, all-around great DTS guy



New Palm OS® 5 APIs



The New APIs

- High-Density Graphics
- Hacking and Notifications
- Native coding
- 802.11
- Sound



High-Density Graphics APIs

- How to make your apps look great
- How the new APIs work
- What to watch out for
- Tools

Good News

- Simplicity and backward compatibility
  - Existing apps run and look great without modification
  - Apps look the same regardless of density
  - Adding support for new features requires minimal changes – just add a high-density bitmap to the family
- Extensible
  - Support for screen densities beyond 2x
  - Support for larger physical screen sizes



Why not adopt existing APIs?

- The same code can run without regard to:
  - Device type
  - Palm OS® version
  - Display density
- Makes existing applications look great with little or no effort

Licensee Adoption of API

- Sony has already abandoned its API in favor of this Palm OS 5 version.
  - Palm OS® 5 -> Sony compatibility layer scheduled for release before Palm OS 5 goes GM.
- Handera will also transition to our API and plans a compatibility layer if necessary.
- Benefit to developers: Code to a single API to run on ALL Palm Powered™ handhelds.

Crucial Facts to Understand

If the display supports it:

- Fonts, buttons etc. are always high density
- All coordinates are 0..159, unless either:
  - The screen gets physically larger, or
  - The app selects native coordinates
  - Most applications will only add new bitmaps
- Normally, no code changes unless:
  - You want to work with individual pixels yourself, or
  - You use offscreen windows

High Density UI

- Fonts, rounded rectangles (e.g. buttons), window frames, etc. will all be drawn at finer resolution, even for old apps
- High density fonts will retain identical metrics so spacing and word-wrapping won't be an issue.
- Applications that are High Density-aware can also draw high density bitmaps just by including one in a bitmap family

Checking for High Density Support

- Maybe you don't need to do anything!
  - You already benefit from high-density UI and bitmaps with no code changes.
- If you do need access to new APIs, check like this:

```
err = FtrGet(sysFtrCreator,
sysFtrNumWinVersion, &version);
```

- If `version` is 4 or greater, the Window Manager supports the high density API.

Coordinate Systems

- Standard – one pixel per standard coordinate
  - Today that's the 160x160 we all know and love
  - Always the default coordinate system
  - High density devices have 2 or more pixels per standard coordinate
  - Form Manager always uses Standard coordinates, preserving compatibility for existing resources (and resource tools, e.g. Constructor)
- Native -- based on physical pixels
  - The blitter always uses native coordinates
  - Direct access to a window's bitmap data is always direct access to the full native resolution

Choosing a Coordinate System

- The Standard coordinate system is the default, even on High Density Devices
- Applications can call WinSetCoordinateSystem() to set the drawing API coordinate system explicitly.
- Generally, to use native coordinates:
  - push draw state
  - modify draw state (scale, etc.)
  - do the drawing
  - pop the draw state
- This technique allows finer resolution for placing graphic primitives (not higher quality primitives)

Window Manager / Blitter interaction

- Every window has an associated draw state that includes a scaling factor
- If the window coordinate system differs from the native coordinate system, the Window Manager performs conversion before calling the blitter.
  - This affects the placement of graphic primitives, not their draw density.
- Regardless of the coordinate system, the blitter uses the density field in the bitmap structure to determine whether to stretch or shrink the bitmap data.

Bitmap Stretching / Shrinking

- When blitting low-density bitmaps to high-density displays, the blitter multiplies each pixel (each pixel becomes 4 pixels, or 9, etc.).
- When blitting high-density bitmaps to low-density displays, the blitter drops pixels.
  - Because of aliasing issues, this can result in poor image quality, so include multiple densities in your bitmap families for best results.

Offscreen Windows

- Old applications continue to work correctly.
- WinCreateOffscreenWindow allocates a low-density bitmap unless the new nativeFormat argument is used.
- BmpCreate and WinCreateBitmapWindow combine to allow the most control over the resulting offscreen window.

Bitmap Selection Algorithm

- Legacy algorithm selected bitmap with greatest bit depth less than or equal to the draw window's bit depth.
- This changes in Palm OS 5:
  - When draw window is 8-bit color, it is better to select a 16-bit image over a grayscale image.
  - 240x320 devices (1.5x density) favor an exact density match.
  - High-density algorithm is more complex:
    - Low-density draw windows favor low-density bitmaps.
    - High-density draw windows favor color domain matches over density matches.



New Tools in SDK 4.0 Update 2

- Updated Constructor
  - To add double-density bitmaps to families
- Updated headers
  - Definitions for new APIs

All the New APIs

- `WinSetCoordinateSystem`
- `WinGetCoordinateSystem`
- `WinScalePoint`
- `WinUnscalePoint`
- `WinScaleRectangle`
- `WinUnscaleRectangle`
- `WinScreenGetAttribute`
- `WinPaintTiledBitmap`
- `WinGetSupportedDensity`
- `BmpGetNextBitmapAnyDensity`
- `BmpGetVersion`
- `BmpGetCompressionType`
- `BmpGetDensity`
- `BmpSetDensity`
- `BmpGetTransparentValue`
- `BmpSetTransparentValue`
- `BmpCreateBitmapV3`
- `EvtGetPenNative`

All the New APIs You're Likely to Use

- (none)
- `WinCreateOffscreenWindow(… nativeFormat…)`
- `WinGetSupportedDensity`
- `BmpCreateBitmapV3` (just creates a header)
- `BmpSetDensity`
- `EvtGetPenNative`, with `WinScalePoint`



The New APIs

- High-Density Graphics
- Hacking and Notifications
- Native coding
- 802.11
- Sound

Hacking and Patching

- Since the OS is native ARM, little-endian and sufficiently different from before, the existing SysSetTrapAddress() is unimplementable
- Most trap patching was done to extend the OS, adding additional features
  - Often was for notification, not a replacement or modification of functionality
- New notifications provide equivalent power for most commonly hacked situations



New Notifications

- Top-used HackMaster hacks were studied
- Hack authors and licensees were surveyed
- Top (implementable) requests:
  - Pen action in silk screen
  - Handling a virtual character
  - App launch and quit notification
  - Idle time detection
  - Watching every single event

New Notifications

- New notifications sent on 5 include:
  - vchr handling before (or instead of) SysHandleEvent
  - Idle time: the system is going into doze mode
  - ProcessPenStroke, interpret strokes in the Graffiti and buttons area
  - application launch & quit
  - keyboard dialog: to override SysKeyboardDialog()
  - Event dequeued
- Definitions are in the 4.0 SDK update
- Negligible performance impact, except last one



Patching Traps

- SysSetTrapAddress returns an error
  - Same with IntlSetRoutineAddress
- Calling SysGetTrapAddress is still fine
- Sustainable patching mechanism coming
  - Will balance security with extensibility



Demo



The New APIs

- High-Density Graphics
- Hacking and Notifications
- Native coding
- 802.11
- Sound



Handling Performance Problems

- Most applications are plenty fast on ARM
- Some portions of some can be slower
  - Depends on many factors
  - Worst case is when an app has a core algorithm that does all the work inside itself
  - Some things may not have been practical before on a Dragonball and now are

Native ARM Coding (cont.)

- These portions can be moved to be native ARM
  - Not the full application, not suited for doing UI
  - Tools are preliminary, gcc-based, but functional
- Easy to make it conditional so the same PRC will work on 68k devices as well as on ARM-based devices
  - App checks `sysFtrNumProcessorID`
  - Calls either 68k, ARM or Simulator version of code

Native ARM Coding (cont.)

- 68k Application built as normal
- If application is running on PACE:
  - Locks resource that contains raw ARM instructions
  - Prepares parameter block (which is the only parameter)
  - Calls a new API, used to call ARM code
- Native code gets the parameter block pointer
  - Has to deal with byte swapping, struct alignment…
  - Can call OS routines; done through PACE to ensure compatibility on future Palm OS® releases

Native ARM: Evaluating the Need

- Very few applications will need it, but some will really benefit
  - Compression, decompression
  - Encryption and decryption
  - Intense graphics (games, multimedia…)
  - Handwriting recognition engines
  - VMs and interpreters
- It is not “pretty”… yet.
- Full tools suite, new runtime and programming paradigm all coming

Native ARM: A How-To guide

1. Find the performance-critical area (don't guess!)
2. Isolate it
3. Convert it to take just 1 parameter
4. Deal with possible changes:
   - Endianness
   - Structure packing
   - Calling back into 68k or into OS
5. Build DLL to test on the Simulator
6. Build with gcc, include into PRC as resource



Demo



The New APIs

- High-Density Graphics
- Hacking and Notifications
- Native coding
- 802.11
- Sound

Palm OS® 802.11b Solutions







Wi Fi™
The Standard for
Wireless Fidelity





Support for the 802.11b standard

- Ability for third parties to build 802.11b solutions (integrated, expansion-card, or sled-based)
- Developers will be able to write network interface modules to the networking stack
- New APIs that will enable the developers to access 802.11b-specific information



802.11b Specific APIs

Developers should be able to GET/SET:

- Preferred Access Points
- Association table
- Network Preference
- Radio Connection Status
- Radio Driver Version
- And more



The New APIs

- High-Density Graphics
- Hacking and Notifications
- Native coding
- 802.11
- Sound

Sound Manager - Overview

- Wave sound resource format
- Raw sampled streaming sound
- Playback of up to 16 channels at a time
  - Each channel could be mono or stereo
  - Recording just 1 channel for now
- 8-bit (signed and unsigned) and 16-bit (signed) data, at arbitrary sampling rates
- APIs for Play, Pause, Stop, Volume…

Sound Resources

- Constructor modified to include wave files
  - Type = ‘wave’
  - Uncompressed or IMA ADPCM
  - 1 or 2 channels, any sampling rate
  - (resource, so 64k max size for now)
- Trivial to play: `SndPlayResource(...)`
- Perfect for games or other applications that want to add small sound resources



Advanced Sound Usage

- Synchronous or asynchronous playback
  - Even with `SndPlayResource`()
- Callbacks can be used for playing and recording
  - Get a buffer; you fill it or empty it
- Can construct a wave resource if you want…



PalmSource®

Conference and Expo

For the web page

- Simulator/Pace usage tips
  - HostControl functions
  - PalmSim.ini
  - -appCreator and other commandline options
  - The database viewer (shift click on resource)
  - The 'Always on top' option
  - PalmTrace.dll - part of Reporter (in search path)
  - Comm ports routing to TCP
  - Run only one sim at a time (else, “End Process”)
  - Don’t mix rom images between .exe files

For the web page (Cont.)

- Simulator/POSE shortcuts
  - Shortcut keys (most also work on Pose 3.3)
    - Control+A — Displays the menu
    - Control+C — Command character
    - Control+D — Confirmation character.
    - Control+E — Displays the application launcher
    - Control+F — Displays the onscreen keyboard
    - Control+I — Jump to the Find Dialog
    - Control+K — Jump to the Calculator app
    - Control+M — Enters a linefeed character
    - Control+N — Jumps to the next field
    - Control+P — Jumps to the previous field
    - Control+R — Does a soft reset
    - Control+Shift+R — Does a hard reset
    - Control+S — Automatic off character
    - Control+Pause — Enters the 68K debugger